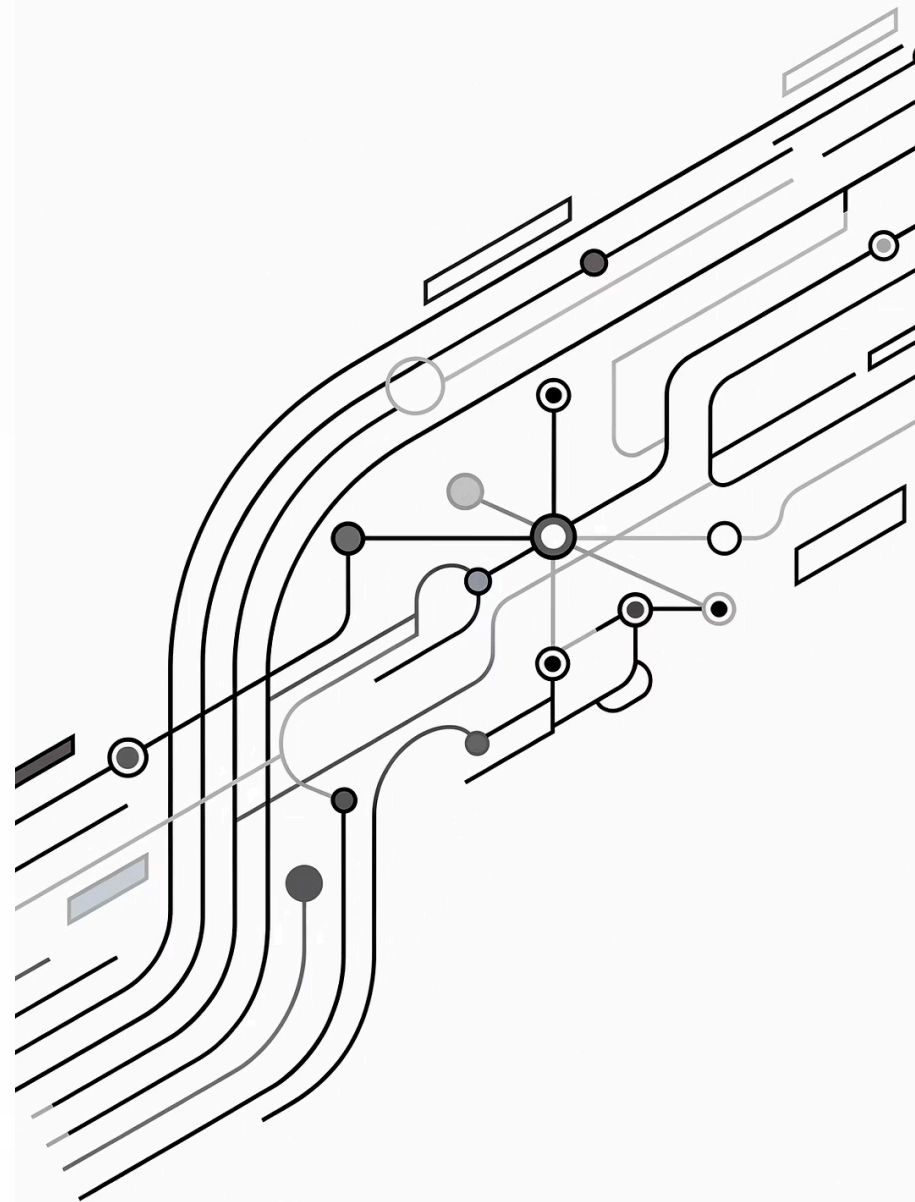


Programmation Réactive avec Spring WebFlux



Plan du module

01

Principes du réactif

Reactive Manifesto, modèle push, back-pressure.

02

Project Reactor

Mono, Flux, schedulers.

03

WebFlux vs MVC

Architecture, cas d'usage, choix technologique.

04

Opérateurs réactifs

map, flatMap, filter, zip, merge, gestion d'erreurs.

05

Tests réactifs

StepVerifier, WebTestClient, bonnes pratiques.

Pourquoi la programmation réactive ?

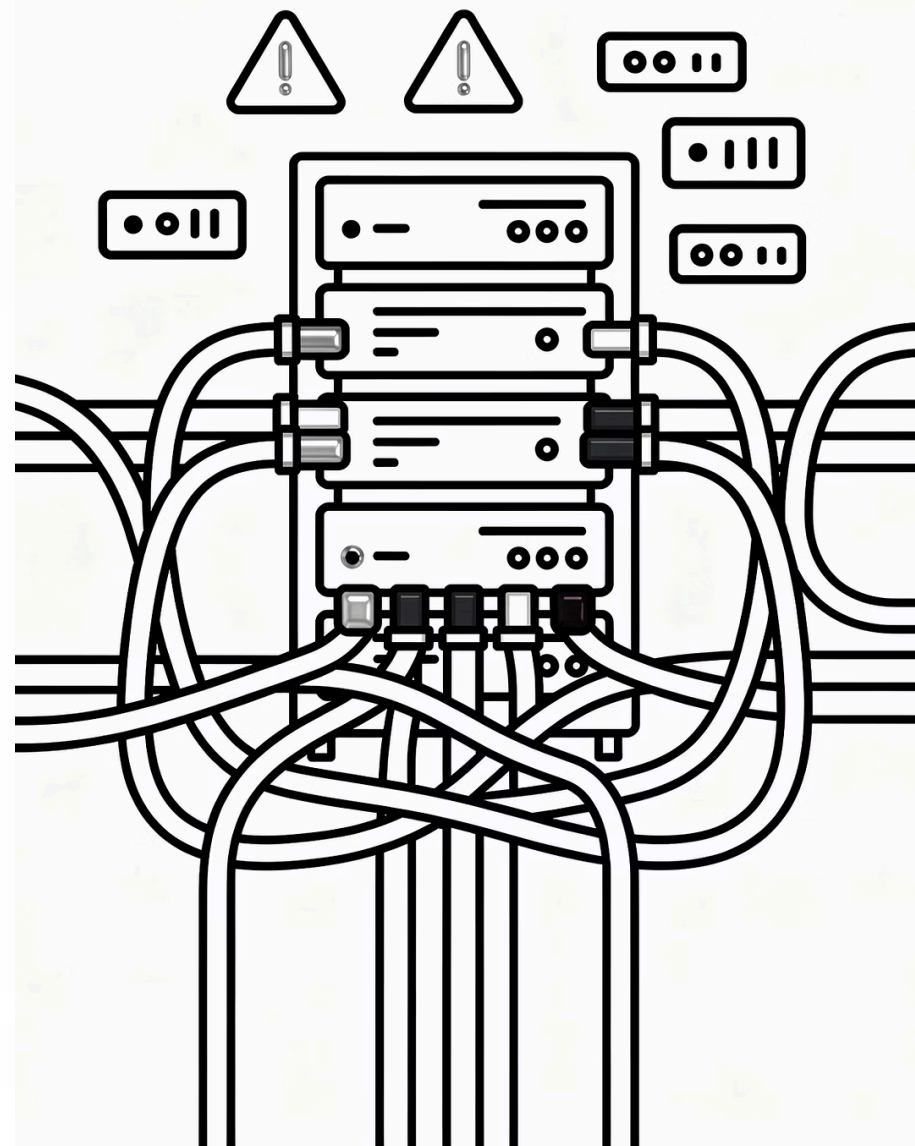
Le modèle **thread-per-request** de Spring MVC est simple, mais présente des limites structurelles sous haute concurrence. Un service qui interroge une BDD et appelle deux API externes bloque son thread à chaque étape — sans faire aucun travail utile.

Modèle bloquant

1 requête = 1 thread occupé.
Thread pool épuisé sous charge.
Scalabilité limitée.

Modèle réactif

Quelques threads, millions d'opérations. I/O non-bloquant.
Back-pressure intégrée. Scalabilité élastique.



Le Reactive Manifesto (2013)

Quatre piliers interdépendants définissent une architecture réactive robuste.



Responsive

Le système répond rapidement en toutes circonstances.



Résilient

Le système reste disponible en cas de défaillance partielle.



Élastique

Le système s'adapte dynamiquement à la charge.

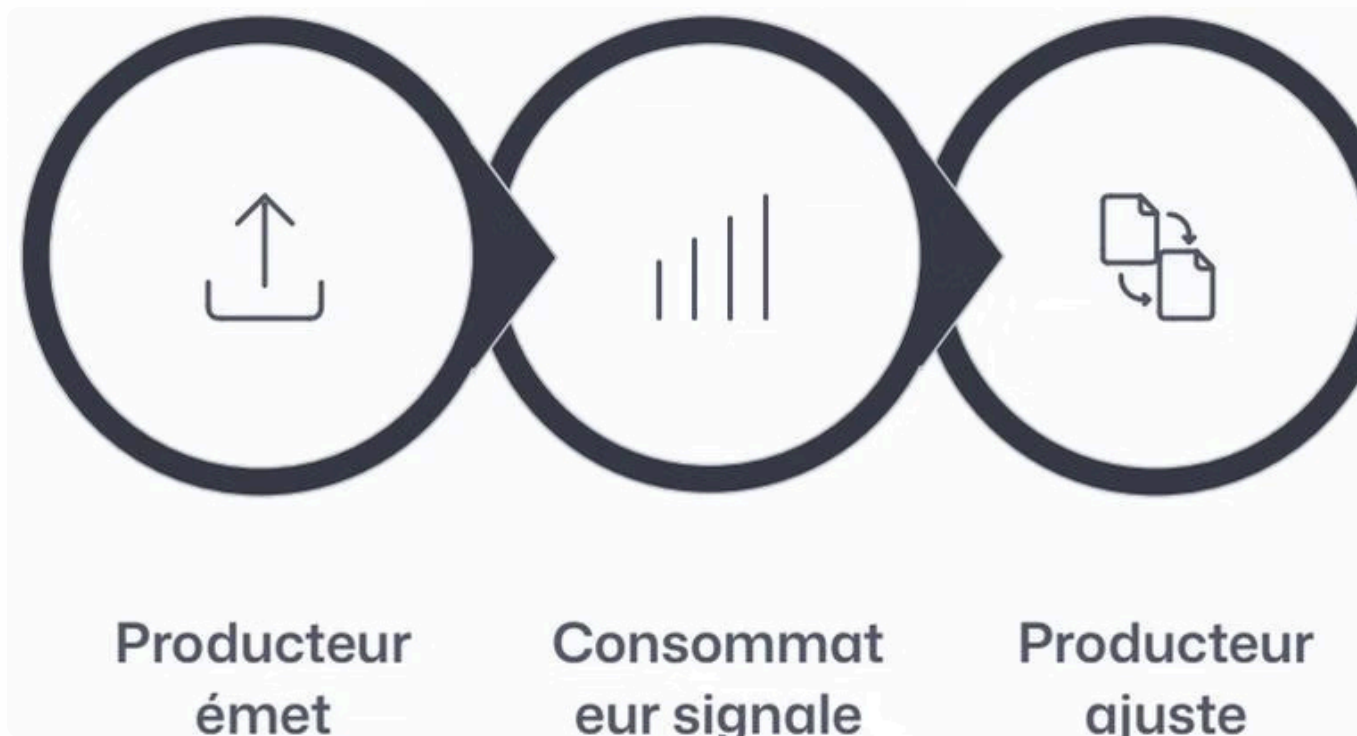


Message-Driven

Communication asynchrone par messages avec back-pressure.

 Le modèle **Message-Driven** est au cœur de Project Reactor — il découple les composants et rend la back-pressure possible.

Le modèle Push et la Back-Pressure



Dans le modèle **push** réactif, le producteur envoie des données dès qu'elles sont disponibles. La **back-pressure** permet au consommateur d'informer le producteur de sa capacité de traitement — évitant ainsi la saturation.

i La spécification **Reactive Streams** (intégrée dans Java 9 via `java.util.concurrent.Flow`) définit quatre interfaces : `Publisher`, `Subscriber`, `Subscription` et `Processor`.

Project Reactor

La bibliothèque réactive de Spring, socle de WebFlux. Elle implémente Reactive Streams et offre une API fluente pour construire des pipelines asynchrones.

Mono<T>

Émet **0 ou 1 élément**, puis se termine ou échoue. Équivalent réactif de `CompletableFuture`.

Flux<T>

Émet **0 à N éléments**, puis se termine ou échoue. Pour les collections, flux continu, SSE.

⚠ Un pipeline réactif est **paresseux** (lazy). Si vous ne souscrivez pas avec `subscribe()`, rien ne s'exécute — source fréquente de bugs chez les débutants.

Mono en pratique

```
// Créer un Mono
Mono<String> mono1 = Mono.just("Hello WebFlux");
Mono<String> mono2 = Mono.empty();
Mono<String> mono3 = Mono.error(new RuntimeException("Erreur"));

// Mono depuis un Callable (lazy)
Mono<User> userMono = Mono.fromCallable(
    () -> userRepository.findById(42));

// Souscrire
mono1.subscribe(
    value -> System.out.println("Valeur : " + value),
    error -> System.err.println("Erreur : " + error),
    () -> System.out.println("Terminé !")
);
```

- ❏ Dans Spring WebFlux, vous retournez rarement un `Mono` souscrit manuellement — le framework gère la souscription lors de chaque requête HTTP.

Flux en pratique

```
Flux<String> flux1 = Flux.just("Spring", "WebFlux", "Reactor");
Flux<User> usersFlux = Flux.fromIterable(userList);
Flux<Integer> range = Flux.range(1, 10);
Flux<Long> ticker = Flux.interval(Duration.ofSeconds(1));

flux1.map(String::toUpperCase)
     .filter(s -> s.startsWith("S"))
     .subscribe(System.out::println);
```

Flux fini

Résultats BDD, listes de ressources, fichiers ligne par ligne.

Flux infini

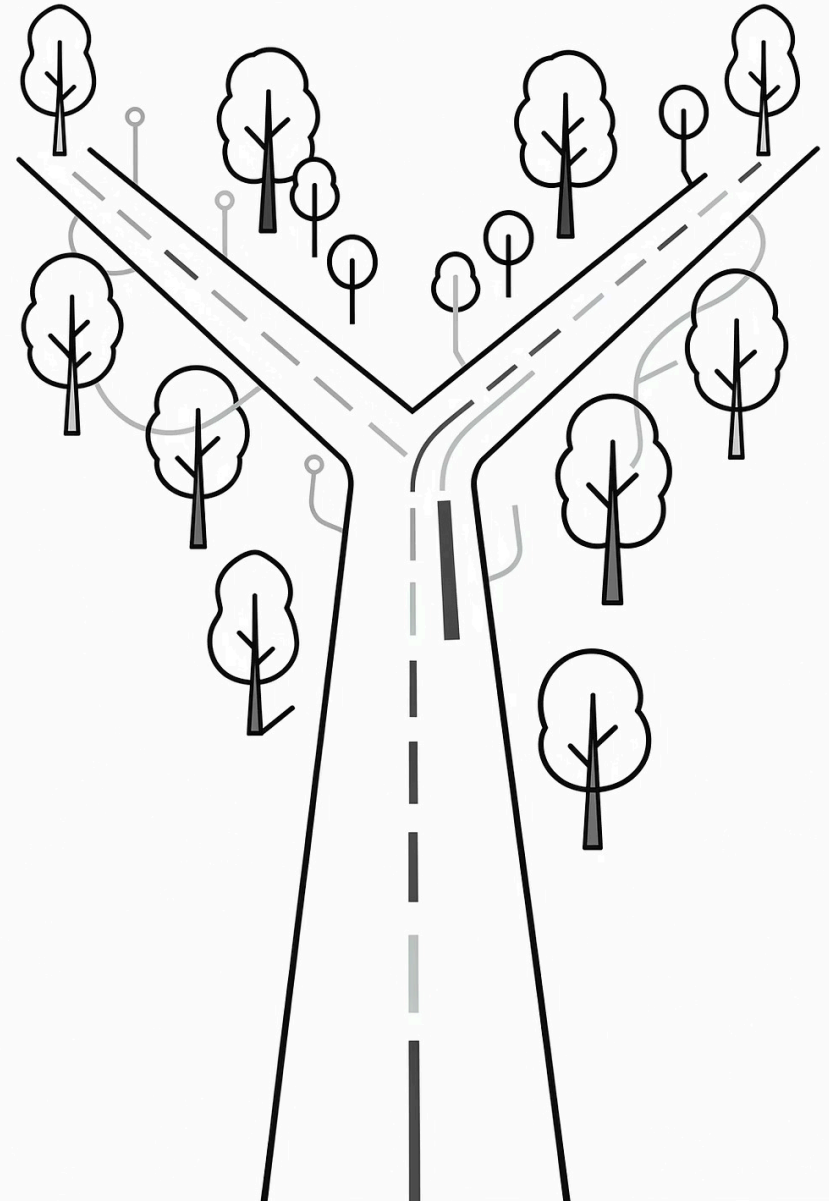
Métriques temps réel, Server-Sent Events, flux Kafka/AMQP.

Flux vide

`Flux.empty()` — cas nominal sans résultat, proprement géré.

WebFlux vs Spring MVC

Deux approches complémentaires qui répondent à des besoins différents. Le choix ne doit pas être dogmatique.



Comparaison architecturale

Spring MVC

- **Concurrence** : Un thread par requête (bloquant)
- **Moteur** : API Servlet (bloquant)
- **API** : Impératif, synchrone
- **Retour** : Objet simple, ResponseEntity
- **Back-pressure** : Absente
- **Usage** : Applications CRUD standard

Spring WebFlux

- **Concurrence** : Boucle d'événements (non-bloquant)
- **Moteur** : Reactor Netty / Undertow
- **API** : Réactif, asynchrone (streams)
- **Retour** : Mono / Flux
- **Back-pressure** : Intégrée
- **Usage** : Streaming, haute concurrence

 Un même projet Spring Boot peut utiliser les deux modules en parallèle. Évitez cependant de mélanger code bloquant et non-bloquant dans les mêmes threads réactifs.

Quand choisir WebFlux ?



Haute concurrence

Des milliers de connexions simultanées avec des opérations I/O intensives (BDD, API HTTP).



Streaming de données

Server-Sent Events, WebSocket, diffusion de fichiers volumineux, flux Kafka.



Orchestration microservices

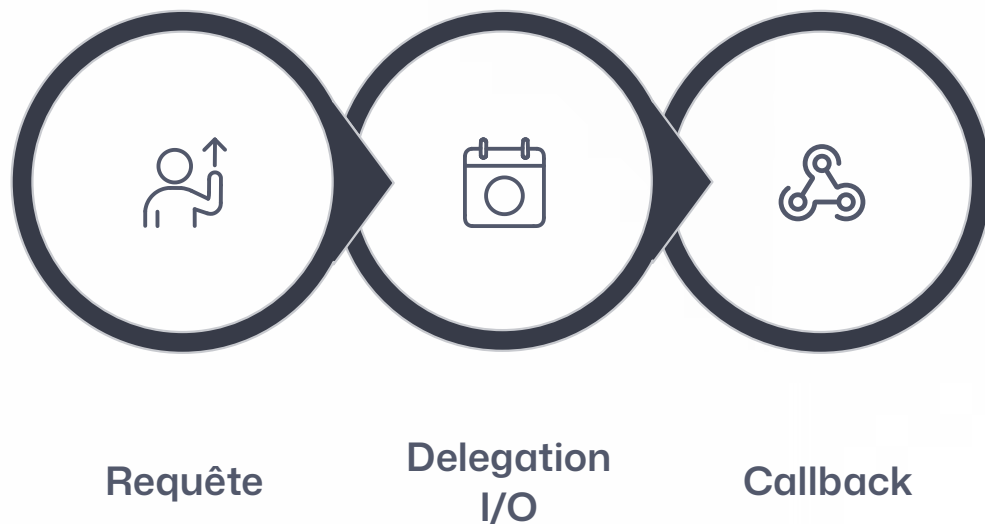
Agrégation de plusieurs appels réseau en parallèle avec WebClient non-bloquant.



Temps réel

Tableaux de bord live, notifications push, systèmes d'événements distribués.

Architecture de l'event loop



WebFlux utilise une **event loop** inspirée de Node.js, implémentée par défaut via **Reactor Netty**. Le nombre de threads est typiquement égal au nombre de cœurs CPU.

⚠ Ces threads ne doivent **jamais être bloqués**. Toute opération bloquante (JDBC, fichiers) doit être déportée sur `Schedulers.boundedElastic()`.

Deux modèles de programmation

Modèle annoté (familier)

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping("/{id}")
    public Mono<User> getUser(
        @PathVariable Long id) {
        return service.findById(id);
    }

    @GetMapping
    public Flux<User> getAllUsers() {
        return service.findAll();
    }
}
```

Modèle fonctionnel (avancé)

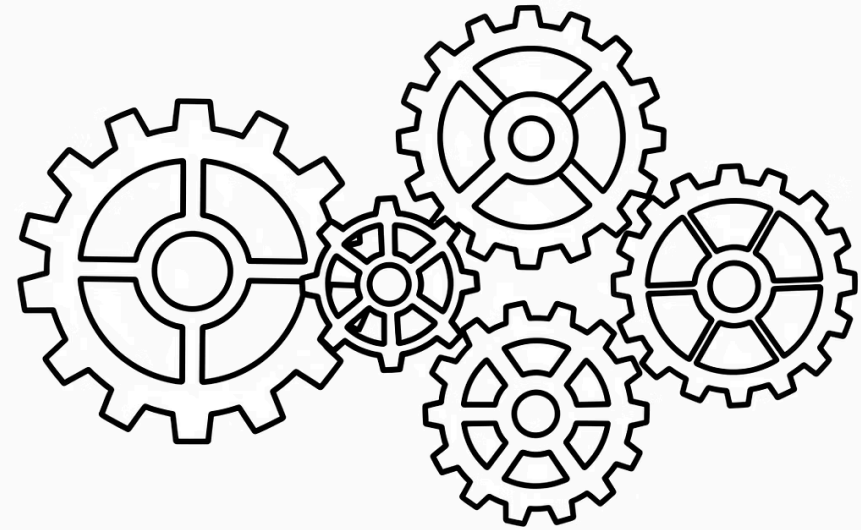
```
@Configuration
public class UserRouter {

    @Bean
    public RouterFunction<ServerResponse>
        route(UserHandler handler) {
        return RouterFunctions.route()
            .GET("/api/users/{id}",
                handler::getUser)
            .GET("/api/users",
                handler::getAllUsers)
            .POST("/api/users",
                handler::create)
            .build();
    }
}
```

Le modèle fonctionnel offre plus de flexibilité pour les routages complexes et les middlewares transverses.

Les opérateurs réactifs fondamentaux

Les opérateurs permettent de transformer, filtrer, combiner et orchestrer des flux de données de manière déclarative.



map, flatMap, concatMap

map – Transformation synchrone

Transforme chaque élément de manière synchrone. Ne crée pas de nouveau Publisher.

```
Flux.just(1, 2, 3).map(n -> n *  
2)  
// émet: 2, 4, 6
```

flatMap – Transformation asynchrone

Transforme chaque élément en un Publisher et aplatit les résultats. **Incontournable** pour les appels BDD/HTTP. L'ordre n'est pas garanti.

```
Flux.just(1, 2, 3)  
  .flatMap(id ->  
    userService.findById(id))  
// appels parallèles, résultats  
entrelacés
```

concatMap – flatMap ordonné

Comme `flatMap` mais préserve l'ordre. Moins performant, utile quand l'ordre des résultats est important.

Opérateurs de filtrage

```
Flux.range(1, 10).filter(n -> n % 2 == 0) // émet: 2, 4, 6, 8, 10
Flux.range(1, 100).take(5)                // émet: 1, 2, 3, 4, 5
Flux.range(1, 10).skip(3)                 // émet: 4, 5, 6, 7, 8, 9, 10
Flux.just("a","b","a","c").distinct()     // émet: a, b, c
Flux.empty().defaultIfEmpty("valeur défaut") // émet: "valeur défaut"
```

```
// switchIfEmpty : flux de substitution si vide
userRepository.findById(id)
    .switchIfEmpty(Mono.error(
        new UserNotFoundException(id)));
```

✔ `switchIfEmpty` remplace élégamment le pattern `Optional.orElseThrow()` du code impératif dans une API REST réactive.

Opérateurs de combinaison

merge

Fusionne N Flux en parallèle, éléments dans l'ordre de publication.

zip

Combine les éléments de N Flux un par un (tuple).
Idéal pour agréger des résultats parallèles.

concat

Concatène N Flux séquentiellement.
Garantit l'ordre.

combineLatest

Émet à chaque nouvelle valeur d'une source, en combinant les dernières valeurs des autres.

Exemple : appels parallèles avec zip

```
Mono<User> userMono =  
    userService.findByld(userId);  
Mono<List<Order>> ordersMono =  
    orderService.findByUserId(userId);  
  
Mono<UserWithOrders> result =  
    Mono.zip(userMono, ordersMono)  
        .map(tuple -> new UserWithOrders(  
            tuple.getT1(),  
            tuple.getT2()));
```

Gestion des erreurs

Stratégies de récupération

```
// Valeur par défaut
flux.onErrorReturn(defaultValue);

// Flux de substitution
flux.onErrorResume(e ->
    fallbackService.getDefault());

// Transformer l'erreur
flux.onErrorMap(
    DatabaseException.class,
    e -> new ServiceException(e));

// Retry avec backoff
flux.retryWhen(
    Retry.backoff(3,
        Duration.ofMillis(100)));
```

Bonnes pratiques

- Gérez les erreurs au plus **proche de la source** avec `onErrorResume`
- Utilisez `onErrorMap` pour transformer les exceptions techniques en exceptions métier
- Préférez `retryWhen` avec **exponential backoff** pour les appels réseau
- Utilisez `@ControllerAdvice` + `@ExceptionHandler` pour la gestion globale

Les Schedulers



`Schedulers.parallel()`

Pool de N threads CPU. Pour les opérations CPU-intensives non-bloquantes. N = nombre de cœurs.



`Schedulers.boundedElastic()`

Recommandé pour les opérations bloquantes (JDBC, fichiers). Pool élastique borné.



`Schedulers.single()`

Thread unique réutilisable. Pour les opérations nécessitant la sérialisation des accès.



Règle d'or : n'utilisez jamais d'opération bloquante sur le thread de l'event loop Netty. Utilisez `subscribeOn(Schedulers.boundedElastic())` pour tout code legacy bloquant.

subscribeOn vs publishOn

subscribeOn(Scheduler)

Affecte le thread sur lequel la **source** est souscrite. S'applique à l'ensemble du pipeline en remontant.

```
Mono.fromCallable(() ->
    blockingRepo.find(id))
    .subscribeOn(
        Schedulers.boundedElastic())
    .map(this::transform);
// source ET map sur boundedElastic
```

publishOn(Scheduler)

Affecte le thread sur lequel les **opérateurs en aval** s'exécutent. Peut être utilisé plusieurs fois.

```
Flux.fromIterable(list)
    .publishOn(Schedulers.parallel())
    .map(this::cpuIntensiveOp)
    .publishOn(
        Schedulers.boundedElastic())
    .flatMap(this::dbWrite);
// deux contextes dans un pipeline
```

Tests réactifs avec StepVerifier

```
// Test d'un Flux avec filtre
@Test
void testUserFlux() {
    Flux<String> flux = Flux.just("Alice", "Bob", "Charlie")
        .filter(name -> name.length() > 3);

    StepVerifier.create(flux)
        .expectNext("Alice")
        .expectNext("Charlie")
        .expectComplete()
        .verify();
}

// Test d'une erreur
@Test
void testErrorHandling() {
    StepVerifier.create(userService.findById(-1L))
        .expectError(UserNotFoundException.class)
        .verify();
}
```

- ✔ Avec `withVirtualTime`, un test qui prendrait 30 secondes en temps réel s'exécute en millisecondes — indispensable pour tester des stratégies de retry ou des flux temporels.

Tests d'intégration avec WebClient

```
@WebFluxTest(UserController.class)
class UserControllerTest {

    @Autowired WebClient webTestClient;
    @MockBean UserService userService;

    @Test
    void getUser_shouldReturnUser() {
        User user = new User(1L, "Alice", "alice@example.com");
        when(userService.findById(1L)).thenReturn(Mono.just(user));

        webTestClient.get()
            .uri("/api/users/1")
            .accept(MediaType.APPLICATION_JSON)
            .exchange()
            .expectStatus().isOk()
            .expectBody(User.class).isEqualTo(user);
    }

    @Test
    void getAllUsers_shouldReturnFlux() {
        when(userService.findAll())
            .thenReturn(Flux.just(
                new User(1L, "Alice"), new User(2L, "Bob")));

        webTestClient.get().uri("/api/users")
            .exchange()
            .expectStatus().isOk()
            .expectBodyList(User.class).hasSize(2);
    }
}
```

Spring Data Reactive : R2DBC

Repository réactif

```
public interface UserRepository
    extends ReactiveCrudRepository
        <User, Long> {

    Flux<User> findByLastName(
        String lastName);
    Mono<User> findByEmail(
        String email);
}

@Table("users")
public class User {
    @Id private Long id;
    private String name;
    private String email;
}
```

Service réactif

```
public Mono<User> findById(Long id) {
    return repo.findById(id)
        .switchIfEmpty(Mono.error(
            new UserNotFoundException(id)));
}

public Flux<User> findAll() {
    return repo.findAll()
        .doOnNext(u ->
            log.debug("User: {}", u));
}
```

⚠ JDBC est bloquant et **incompatible** avec WebFlux sur l'event loop. Migrez vers R2DBC ou isolez sur `Schedulers.boundedElastic()`.

WebClient & Server-Sent Events

WebClient – client HTTP réactif

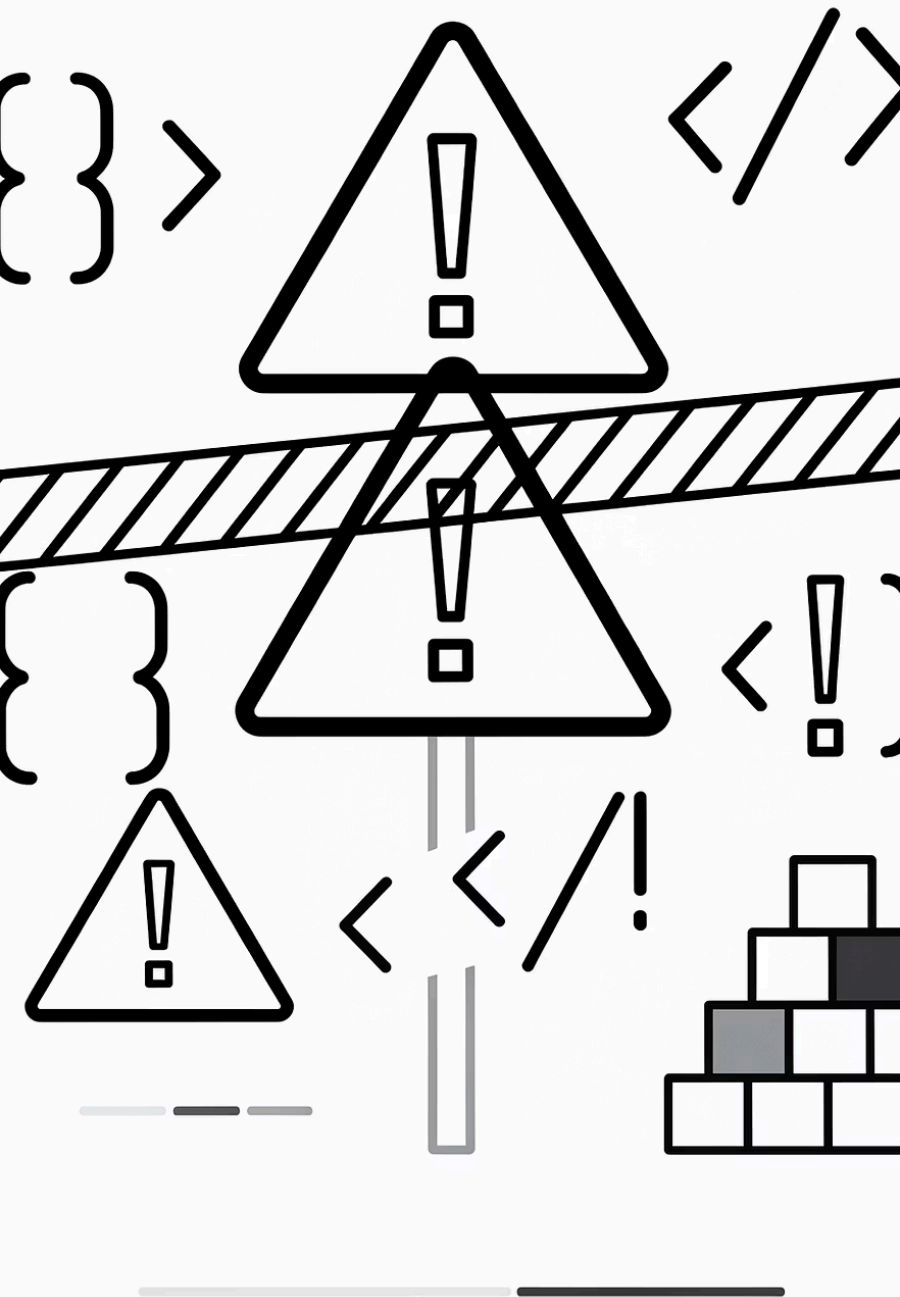
```
public Mono<Product> getProduct(Long id) {
    return webClient.get()
        .uri("/products/{id}", id)
        .retrieve()
        .onStatus(HttpStatus::is4xxClientError,
            res -> Mono.error(
                new ProductNotFoundException(id)))
        .bodyToMono(Product.class)
        .retryWhen(Retry.backoff(3,
            Duration.ofMillis(500)));
}

// Appels parallèles avec zip
public Mono<ProductBundle> getBundle(
    Long id1, Long id2) {
    return Mono.zip(
        getProduct(id1), getProduct(id2),
        ProductBundle::new);
}
```

Server-Sent Events

```
@GetMapping(produces =
    MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<String> streamEvents() {
    return Flux.interval(
        Duration.ofSeconds(1))
        .map(tick -> "Event #" + tick)
        .take(100);
}
```

 Le client JavaScript utilise `new EventSource('/api/events')`. WebFlux gère automatiquement la déconnexion et l'annulation du Flux — la back-pressure est intégrée.



Anti-patterns à éviter

✗ Utiliser `.block()` dans un pipeline

Bloque l'event loop Netty et provoque une `IllegalArgumentException`. Utilisez `flatMap` pour composer des `Mono`.

✗ Imbriquer des `subscribe()`

Crée un pipeline "fire and forget" déconnecté du pipeline principal. Les erreurs ne se propagent pas.

✗ Mélanger code bloquant et réactif

Appeler JDBC ou `Thread.sleep()` sans `Schedulers.boundedElastic()` dégrade drastiquement les performances.

✗ Ignorer les erreurs

Ne pas gérer `onError` laisse les exceptions terminer le flux silencieusement. Gérez toujours avec `onErrorResume` ou `onErrorMap`.